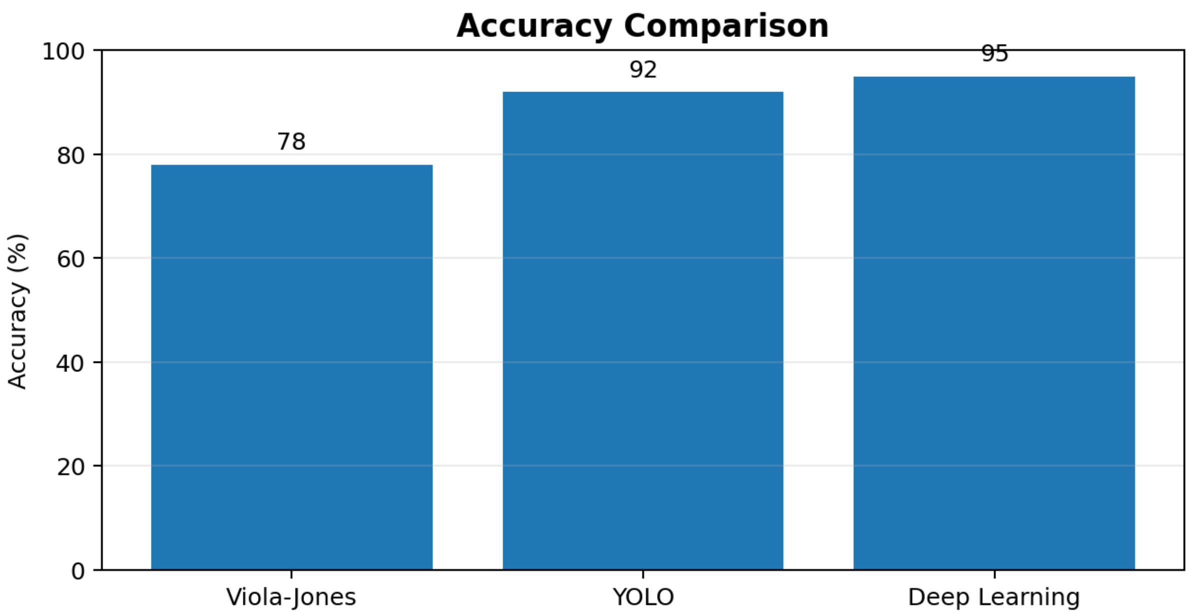
CO4_AT1 – DATA INTERPRETATION

1. Real-Time Detection System

A surveillance system has to detect objects or events continuously. The supplied data compares three methods: Viola-Jones, YOLO and Deep Learning. Their accuracies are 78%, 92% and 95%, while their speeds are listed as Fast, Moderate and Slow respectively. The main decision is therefore not based on accuracy alone. A surveillance system working under strict latency constraints must respond quickly while still maintaining acceptable detection accuracy.

Given Data

Method	Accuracy	Speed
Viola-Jones	78%	Fast
YOLO	92%	Moderate
Deep Learning	95%	Slow



The chart clearly shows that Deep Learning has the highest accuracy, followed by YOLO and Viola-Jones. However, the supplied speed information shows the opposite trend: Viola-Jones is fast, YOLO is moderate, and Deep Learning is slow. This creates a practical accuracy–latency trade-off.

Evaluation of Viola-Jones

Viola-Jones provides 78% accuracy and is identified as Fast. Its strongest advantage is low processing delay. This is useful when the surveillance application has very limited computing resources or when an immediate response is more important than maximum detection accuracy. However, 78% accuracy also means that more detections may be incorrect or missed compared with the other two methods. Therefore, it is suitable only when the application can tolerate lower accuracy.

Evaluation of YOLO

YOLO provides 92% accuracy with Moderate speed. It offers a much higher accuracy than Viola-Jones while avoiding the Slow speed associated with the Deep Learning option in the supplied table. Therefore, YOLO represents a strong compromise between detection quality and response time. For a practical surveillance system, a moderate-speed method with 92% accuracy can be preferable when the system needs reliable detection but cannot accept the latency of the slowest method.

Evaluation of Deep Learning

Deep Learning has the highest accuracy of 95%, but its speed is listed as Slow. This makes it attractive when detection accuracy is the primary requirement and the deployment environment can tolerate higher processing delay. Under strict latency constraints, however, its slow speed becomes a major disadvantage. A surveillance system may need to react immediately to an event, so the extra accuracy may not justify the additional delay.

Final Selection

For the specific condition stated in the question—strict latency constraints while maintaining acceptable accuracy—YOLO should be selected. Viola-Jones is faster, but its 78% accuracy is comparatively low. Deep Learning gives the best accuracy, but its slow speed conflicts with the latency requirement. YOLO, with 92% accuracy and moderate speed, provides the most balanced choice.

- If latency is extremely strict and some accuracy loss is acceptable: Viola-Jones can be considered.
- If both accuracy and response time are important: YOLO is the preferred choice.
- If maximum accuracy is more important than latency: Deep Learning is preferable.
- For the stated deployment condition, YOLO gives the best practical compromise.

Conclusion

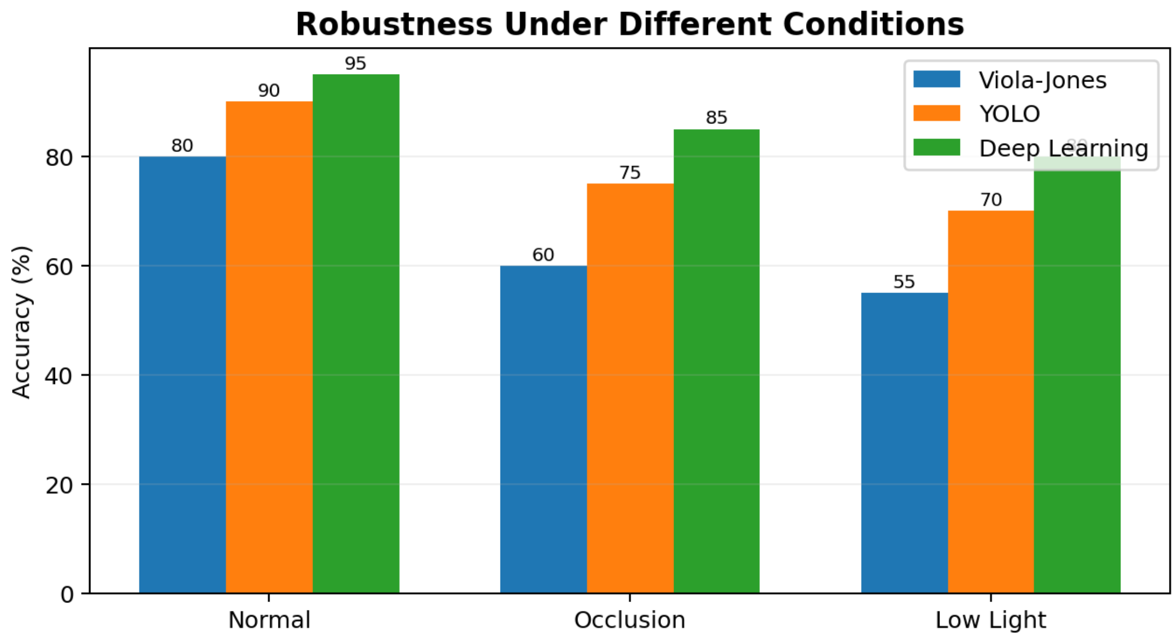
Thus, the decision should be based on the deployment requirement rather than accuracy alone. YOLO is the recommended method for the stated surveillance scenario because it provides 92% accuracy while retaining moderate processing speed. It offers a practical middle ground between the fast but less accurate Viola-Jones method and the highly accurate but slow Deep Learning method.

2. Robust Object Detection

Robustness means the ability of an object detection method to maintain good performance when operating conditions change. The supplied data evaluates Viola-Jones, YOLO and Deep Learning under three conditions: Normal, Occlusion and Low Light. The results show how each method behaves when the visual scene becomes more difficult.

Given Data

Method	Normal	Occlusion	Low Light
Viola-Jones	80%	60%	55%
YOLO	90%	75%	70%
Deep Learning	95%	85%	80%



Under normal conditions, Deep Learning has the highest accuracy at 95%, YOLO follows at 90%, and Viola-Jones records 80%. When occlusion is introduced, the accuracies fall to 85%, 75% and 60% respectively. Under low-light conditions, they fall further to 80%, 70% and 55%. This indicates that all three methods are affected by difficult conditions, but the reduction is smallest for Deep Learning.

Normal Conditions

In a normal scene, Deep Learning achieves 95% accuracy, which is the best result. YOLO also performs strongly at 90%, while Viola-Jones reaches 80%. Therefore, even before environmental difficulties are introduced, the ranking is Deep Learning, YOLO and Viola-Jones.

Occlusion and Low-Light Conditions

Occlusion makes part of an object hidden. The supplied results show Deep Learning at 85%, YOLO at 75%, and Viola-Jones at 60%. Deep Learning therefore retains the highest accuracy when objects are partially blocked. In low light, Deep Learning again performs best at 80%, followed by YOLO at 70% and Viola-Jones at 55%.

Overall Comparison

Method	**Normal**	**Occlusion**	**Low Light**	**Overall Observation**
Viola-Jones	80%	60%	55%	Lowest robustness
YOLO	90%	75%	70%	Good compromise
Deep Learning	95%	85%	80%	Best overall robustness

Deep Learning provides the best performance in every condition listed in the question. It also shows the strongest ability to retain accuracy as conditions become difficult. YOLO is the second-best method and provides a useful middle option. Viola-Jones has the lowest results, especially in occlusion and low light.

Trade-offs

- Deep Learning: highest accuracy and best robustness, but its main practical trade-off is usually greater computational demand and processing requirements.
- YOLO: lower accuracy than Deep Learning but still strong, making it a practical compromise when resources or response time matter.
- Viola-Jones: simple and useful in less demanding settings, but the supplied results show a significant loss of accuracy under occlusion and low light.

Conclusion

Based strictly on the supplied performance table, Deep Learning is the best overall method for robust object detection because it achieves the highest accuracy under normal, occluded and low-light conditions. YOLO is the next-best alternative when a balance is needed, while Viola-Jones is the least robust among the three for the tested conditions.

3. Optical Flow Analysis

Optical flow is used to represent apparent motion in an image sequence. The supplied question describes two methods rather than providing numerical accuracy values. Method A produces stable motion vectors but misses small motions. Method B detects fine motion but produces noisy vectors. The comparison therefore has to consider accuracy, stability, noise sensitivity and the requirements of the application.

Method A – Stable Motion Vectors

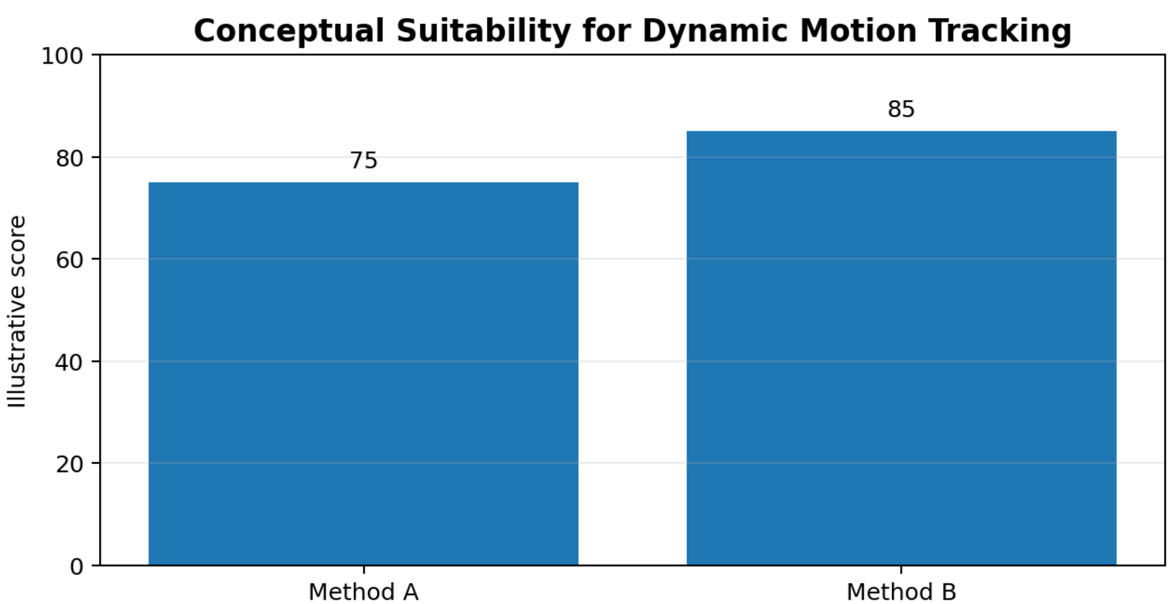
Method A has the major advantage of stability. Stable vectors provide a smoother and more consistent representation of large or dominant motion. This can be useful in a dynamic scene where the tracking system needs reliable direction and movement information without excessive fluctuations. However, Method A misses small motions. Therefore, objects or subtle movements that occupy only a small region may not be detected accurately.

- Strength: stable and consistent motion vectors.
- Strength: lower sensitivity to random vector fluctuations.
- Limitation: small or fine motions may be missed.
- Best suited to applications where smooth and stable motion is more important than tiny movements.

Method B – Fine Motion Detection

Method B can detect fine motion, which is a significant advantage in scenes containing small or subtle movements. However, it produces noisy vectors. Noise can create false or unstable motion directions and may make tracking less reliable if the application requires smooth trajectories. Therefore, Method B has stronger sensitivity to small movements but weaker stability.

- Strength: detects fine and small motion.
- Strength: useful when subtle movement is important.
- Limitation: noisy vectors can reduce tracking stability.
- Best suited to applications where detecting small motion is more important than perfectly smooth vectors.



Comparison

Criterion	Method A	Method B
Motion stability	High	Lower due to noise
Fine-motion detection	Lower; may miss small motion	High
Noise sensitivity	Lower	Higher
Tracking smoothness	Better	May fluctuate
Main advantage	Stable vectors	Fine motion detection
Main limitation	Misses small motions	Noisy vectors

Which Method is More Suitable?

There is no universal winner because the correct choice depends on the application requirement. If the goal is stable motion tracking in a dynamic scene, Method A is generally more suitable because stable vectors reduce fluctuations and make the estimated motion easier to follow. If the scene specifically contains small movements that must not be missed, Method B may be more suitable, provided that noise can be filtered or otherwise managed.

For a general dynamic-scene tracking application where accuracy means both correct movement estimation and stable trajectories, Method A is the safer choice. Its limitation—missing small motions—can be acceptable when the application mainly tracks larger object movement. Method B becomes preferable for fine-grained motion analysis, such as detecting subtle changes, but the noise must be handled carefully.

Conclusion

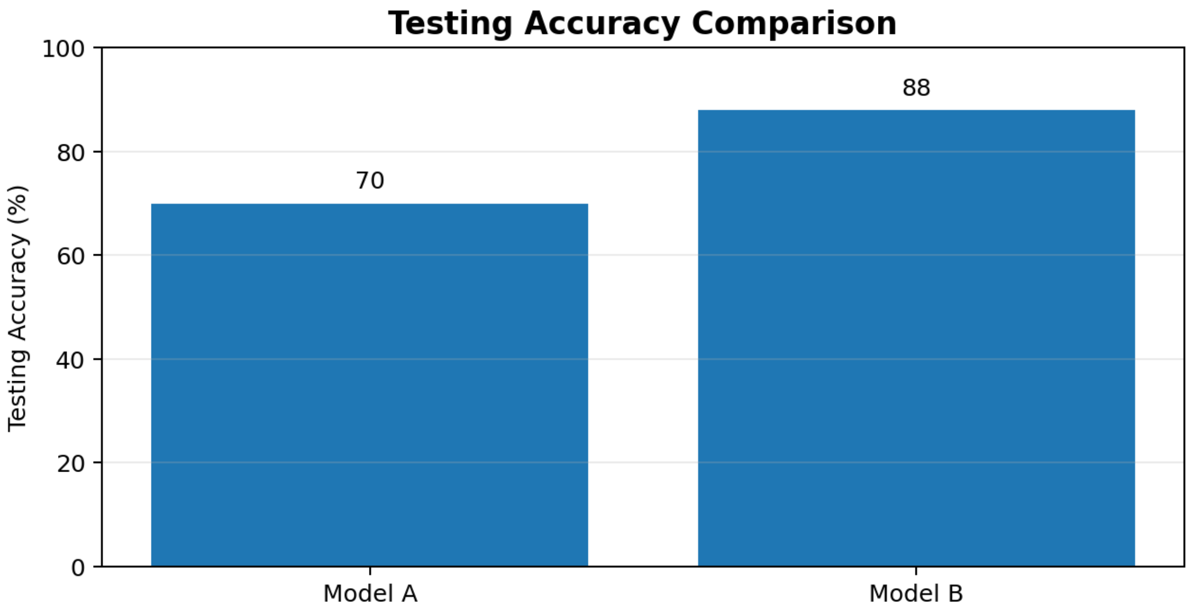
Method A is recommended for general dynamic motion tracking when stability and consistent tracking are the primary requirements. Method B is recommended when detecting small motion is the most important requirement. Thus, the final selection should be made according to whether the application values stable vectors or fine-motion sensitivity more strongly.

4. Model Generalization and Overfitting

The supplied data compares two models using training accuracy and testing accuracy. Model A has 98% training accuracy but only 70% testing accuracy. Model B has 90% training accuracy and 88% testing accuracy. The purpose is to determine which model is more reliable for real-world deployment and to explain the generalization and overfitting risk.

Given Data

Model	Training Accuracy	Testing Accuracy
A	98%	70%
B	90%	88%



Understanding the Accuracy Gap

A model that performs extremely well on training data but much worse on unseen testing data may have learned the training examples too specifically. This is known as overfitting. Model A shows a large difference between training and testing accuracy: 98% − 70% = 28 percentage points. This large gap indicates a strong risk that the model does not generalize well to unseen data.

Model B has a much smaller gap: 90% − 88% = 2 percentage points. Its training and testing results are therefore much closer. This indicates stronger generalization because the model maintains nearly the same performance when it is evaluated on data that was not used for training.

Model A Evaluation

Model A appears excellent if only the training result is considered. However, the 70% testing accuracy is much lower. For real-world deployment, this is a concern because actual data can differ from the training examples. The model may fail more often than its 98% training result suggests. Therefore, Model A has a high overfitting risk and lower reliability for unseen situations.

Model B Evaluation

Model B has 90% training accuracy and 88% testing accuracy. Although its training accuracy is lower than Model A, its testing accuracy is much higher. The small two-percentage-point gap suggests that Model B has learned patterns that transfer better from training data to unseen data. This is the key property required for reliable real-world deployment.

Generalization Comparison

Factor	Model A	Model B
Training accuracy	98%	90%
Testing accuracy	70%	88%
Train–test gap	28 percentage points	2 percentage points
Overfitting risk	High	Low
Generalization	Weak	Strong
Deployment reliability	Lower	Higher

Selection for Real-World Deployment

Model B should be selected for real-world deployment. The objective is not simply to obtain the highest training accuracy; the important requirement is good performance on unseen data. Model B achieves 88% testing accuracy, compared with only 70% for Model A. Its small train–test gap also indicates that its performance is more consistent.

- Model A: very high training accuracy but poor testing accuracy.
- Model A: 28-point gap indicates substantial overfitting risk.
- Model B: slightly lower training accuracy but much better testing accuracy.
- Model B: only 2-point gap indicates stronger generalization.
- Final choice: Model B for reliable real-world deployment.

Conclusion

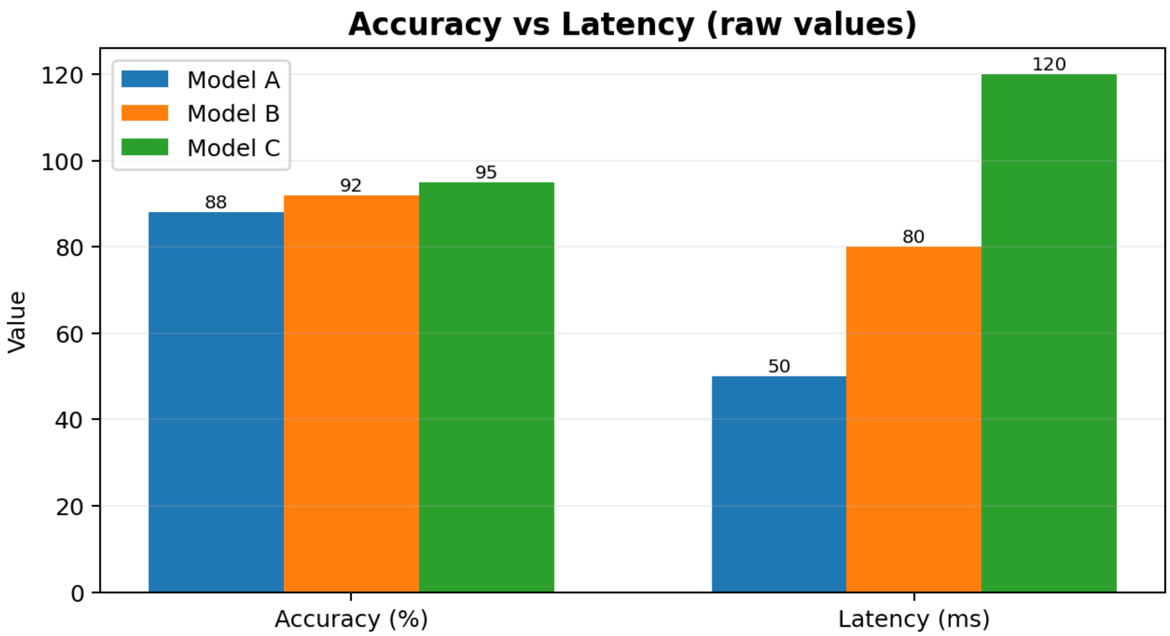
Model B is more reliable because it generalizes better to unseen data. Model A demonstrates a classic overfitting pattern: excellent training performance but a large drop during testing. Therefore, the model with the smaller train–test gap and higher testing accuracy is the better deployment choice.

5. System Performance under Constraints

The supplied data compares three models using three practical deployment factors: accuracy, latency and cost. Model A has 88% accuracy, 50 ms latency and low cost. Model B has 92% accuracy, 80 ms latency and moderate cost. Model C has 95% accuracy, 120 ms latency and high cost. The task is to identify the best balance for a real-time application.

Given Data

Model	Accuracy	Latency	Cost
A	88%	50 ms	Low
B	92%	80 ms	Moderate
C	95%	120 ms	High



The table shows a clear trade-off. As the model moves from A to B to C, accuracy increases from 88% to 92% to 95%. At the same time, latency increases from 50 ms to 80 ms to 120 ms, and cost increases from Low to Moderate to High. Therefore, selecting the highest-accuracy model automatically would ignore important deployment constraints.

Model A

Model A has the lowest accuracy among the three, at 88%, but it has the lowest latency of 50 ms and low cost. This makes it attractive for applications where rapid response and budget efficiency are very important. Its main limitation is lower accuracy compared with Models B and C.

Model B

Model B provides 92% accuracy with 80 ms latency and moderate cost. It improves accuracy by 4 percentage points over Model A while increasing latency by 30 ms. It also costs more than Model A, but less than the high-cost Model C. This makes Model B a balanced middle option.

Model C

Model C gives the highest accuracy, 95%, but it also has the highest latency, 120 ms, and high cost. It may be appropriate when maximum accuracy is the main objective and the application can tolerate

Overall Comparison

Model	Accuracy	Latency	Cost	Main Advantage	Main Limitation
A	88%	50 ms	Low	Fast and economical	Lowest accuracy
B	92%	80 ms	Moderate	Balanced performance	Higher latency/cost than A
C	95%	120 ms	High	Highest accuracy	Slowest and most expensive

Best Balance for Real-Time Application

For a real-time application, Model B provides the best overall balance among the three. Model A is faster and cheaper, but its 88% accuracy is lower. Model C is more accurate, but its 120 ms latency and high cost make it less attractive when response time and practical deployment cost matter. Model B sits between the two: 92% accuracy, 80 ms latency and moderate cost.

The choice can change if the deployment constraints change. If the application has extremely strict latency and budget limits, Model A may be selected. If the application demands maximum accuracy and can tolerate 120 ms latency and high cost, Model C may be selected. However, under the question's general requirement of balancing accuracy, latency and cost for real-time use, Model B is the most reasonable choice.

Decision Summary

- Model A: best for minimum latency and low cost.
- Model B: best overall balance of accuracy, latency and cost.
- Model C: best when maximum accuracy is more important than latency and cost.
- Recommended for the stated real-time balance requirement: Model B.

Conclusion

Model B should be selected for the real-time application because it provides a practical compromise. Its 92% accuracy is substantially better than Model A, while its 80 ms latency and moderate cost are more manageable than Model C's 120 ms latency and high cost. Therefore, Model B achieves the best balance under practical deployment constraints.